

Quality Assurance Report

Pharmacy Management System — Backend API

Subject	pharmacy-backend — Spring Boot 4.1.0 REST API (Java 21)
Assessed	24 August 2026
Revision	Rev. 2 — coverage extended to the full codebase. pharmacy-backend @ main, 127 main source files, 0 pre-existing tests
Activity	Risk-based static review and construction of an automated regression suite
Result	338 tests written — 326 passing, 12 reproducing 6 open defects

Executive summary

The application was delivered with no automated tests beyond an empty Spring context-load check, and that check could not run at all — it required a live PostgreSQL instance and a real JWT secret in the environment. Every business rule in the system was therefore unverified.

This engagement built a 338-test suite. Effort was concentrated first on the areas where a defect causes real-world harm — dispensing, controlled substances, authentication, role-based authorisation — and then extended to every remaining service, controller and security component. The code is generally well structured: rules are enforced in the service layer rather than scattered through controllers, prices are computed server-side and never accepted from the client, and passwords are hashed correctly. Most rules that are implemented are implemented properly.

Six defects were found. Two are significant: one business rule is documented but never enforced at the point it matters, and a single line in the global error handler causes every authorisation denial in the system to be reported as a server crash.

Headline finding — DEFECT-06. The catch-all `@ExceptionHandler(Exception.class)` in `GlobalExceptionHandler` intercepts Spring Security's `AuthorizationDeniedException` before the framework can translate it. Access is still correctly *denied*, but every blocked request returns **500 “An unexpected error occurred”** instead of 403, logs a stack trace at ERROR, and bypasses `JwtAccessDeniedHandler` entirely — a class written for this exact purpose that is currently unreachable. This affects Rules 2, 11, 12, 15 and 16, i.e. the whole permission model. The fix is roughly four lines.

Defects at a glance

ID	Severity	Area	Finding
DEFECT-01	High	Dispensing	Rule 11 not enforced at point of sale — unverified controlled-substance stock can be dispensed
DEFECT-06	High	Authorisation	Every @PreAuthorize denial returns 500 instead of 403; access-denied handler is dead code
DEFECT-02	Medium	Authentication	Login reveals that a deactivated account exists, before any password check
DEFECT-03	Medium	Staff admin	Pharmacist licence uniqueness checked on create but not on update
DEFECT-04	Medium	Procurement	markDelivered has no state guard — a cancelled order can be marked Received
DEFECT-05	Medium	Error handling	Malformed input (bad UUID, unknown enum) returns 500 rather than 400

No defect found in this review permits an unauthorised user to perform a privileged action. The failures are of contract, enforcement completeness, and information disclosure. Twelve further observations — decisions to confirm rather than faults to fix — are listed in §4.

1. Scope and approach

1.1 What was assessed

The backend exposes 11 controllers over a domain of drugs, batches, prescriptions, sales, staff, suppliers, purchase orders and an audit log, secured by stateless JWT with role-based access for Pharmacist, Technician and Admin. Every controller, service and security component now carries tests. Testing them all to the same *depth* would spend the same effort on a supplier address field as on the code that hands a controlled substance to a patient, so depth was allocated by consequence of failure:

Priority	Area	Why it was prioritised
1	Sale / dispensing	The only path that moves stock, takes money and releases controlled drugs. Rules 1, 3, 4, 5 and 15 must all hold simultaneously.
2	Prescription approval	Gatekeeper for dispensing. A weak state machine here means a prescription can be approved twice or resurrected after rejection.
3	Authentication & accounts	Password handling, the Rule 13 forced reset, and account deactivation. Failures are silent and exploitable.
4	Authorisation	Rules 2, 11, 12, 15 exist only as @PreAuthorize annotations. Nothing below the controller re-checks them.
5	Identity resolution	The JWT filter decides who the caller is on every request, and reloads the Staff row so deactivation takes effect before the token expires.
6	Batch / expiry	Rule 5 is absolute: expired stock must never be dispensed. Boundary behaviour at exactly today's date decides whether usable stock is wrongly blocked.
7	Input validation	Cheapest place to stop bad data, and the cross-field rules are the ones most likely to rot unnoticed.
8	Enum ↔ schema mapping	Two enums map to database strings that differ from the constant name. A mismatch is a runtime constraint violation, not a compile error.
9	Reference data	Customer, doctor and supplier records. Lower stakes, but each carries a natural-key uniqueness rule that must not fire against the record itself on update.

1.2 Method

Source review of all 127 main files, followed by construction of executable tests for the behaviour each rule claims. Where the code and its own documentation disagreed, the documented requirement was treated as the specification and a failing test was written against it — that is how DEFECT-01 was found.

Service-layer tests use mocked repositories so a rule can be violated in isolation. Running them against a real database would let UNIQUE and CHECK constraints mask whether the service itself enforces anything. Controller tests take the opposite approach and load the real security chain, because an access-control assertion against a mocked entry point proves nothing.

Tests are written to the specification, not to the implementation — but the specification is not assumed to be the tester's first guess. One test initially failed because it asserted that a punctuation-only string should be rejected as a phone number; on investigation the code was right and the expectation was wrong. The test was corrected and the underlying permissiveness recorded as OBS-11 rather than quietly dropped.

1.3 Explicitly out of scope

- **Database migrations.** The Flyway scripts are PostgreSQL-specific; the suite runs on H2 with Hibernate schema generation. Drift between the JPA entities and `V1__init_schema_and_seed_data.sql` would not be caught. Partially mitigated — see §5 and OBS-01.
- **Load, soak and concurrency testing.** One concurrency risk is noted at OBS-08 but was not exercised.
- **Penetration testing.** DEFECT-02 was found by reading the login flow, not by an offensive assessment.
- **The 11 MapStruct mappers.** They are unreferenced dead code (OBS-12). Writing tests for them would imply they are load-bearing; the correct action is to wire them in or delete them.
- **Trivial and generated code** — `OpenApiConfig` (Swagger metadata), the application entry point, and pure data-carrier records exercised through their parents.
- **The frontend** and any deployment or infrastructure configuration.

2. The test suite

20 test classes, 338 tests. The suite runs in under a minute on a laptop with no database, no network and no environment variables required.

Test class	Tests	Coverage focus
<code>RequestValidationTest</code>	47	Every request DTO, including the cross-field <code>@AssertTrue</code> rules on staff creation and update
<code>DomainModelTest</code>	29	Enum ↔ CHECK-constraint alignment, expiry boundary, <code>@PrePersist</code> defaults
<code>AdminControllersSecurityTest</code>	27	AuditLog, Staff, PurchaseOrder, Supplier — Rules 7, 13, 16, 17
<code>ClinicalControllersSecurityTest</code>	27	Prescription, Batch, Customer, Doctor — Rules 2, 5, 11
<code>SaleServiceImplTest</code>	22	Rules 1, 3, 4, 5, 11, 15 — approval gate, double-dispense, expiry boundary, oversell, split cashier/dispenser, server-side pricing, audit notes
<code>SecurityComponentsTest</code>	21	Role→authority mapping, Staff reload per request, deactivation taking effect before token expiry, non-Bearer and forged headers
<code>PrescriptionServiceImplTest</code>	17	Rule 2 role gate, full approve/reject state machine, note appending, audit trail
<code>DirectoryServicesTest</code>	17	Customer, Doctor, Supplier — natural-key uniqueness including the self-collision case on update

Test class	Tests	Coverage focus
StaffServiceImplTest	14	Rule 13 forced reset, BCrypt-only persistence, one-time temp password, duplicate guards
AuditLogServiceImplTest	13	Rule 7 — attribution, DB-value translation for every action type, filters
BatchServiceImplTest	13	Rule 11 verification role gate, receipt stamping, expiry-window reporting
PurchaseOrderServiceImplTest	13	Rules 16, 17 — order lifecycle, overdue boundaries, awaiting-verification de-duplication and ordering
JwtTokenProviderImplTest	11	Claim contents, foreign signature, tampered payload, expiry, weak-key rejection
DrugControllerSecurityTest	10	Rule 12 Admin-only enforcement, 401 vs 403, status-code mapping
AuthServiceImplTest	10	Login outcomes, generic failure messages, password change, reset-flag clearing
AuthControllerTest	9	Public login route, authenticated change-password, self-scoped by token
SaleControllerSecurityTest	9	Rule 15 at HTTP boundary, identity from token not body, 422 mapping
DrugServiceImplTest	9	Catalog defaults, narrowness of each Rule 12 update path
ApiResponseTest	7	The uniform response envelope — success/error flags, ISO-8601 stamping
PharmacyApplicationTests	1	Spring context loads (now runnable without a database)
Total	338	<i>326 passing · 12 tagged known-defect</i>

2.1 How defects are recorded as tests

Each defect has at least one test that asserts the *required* behaviour, so it fails against the current code. Those tests carry `@Tag("known-defect")` and are excluded from the default build. This keeps two signals separate: a green `mvn test` means “no regressions”, while the tagged run is a live, executable defect list that turns green as fixes land.

Regression gate — expected to pass:

```
mvnw test
```

Open-defect reproduction — expected to fail, 12 failures:

```
mvnw test -Dgroups=known-defect -DexcludedGroups=
```

When a defect is fixed, its test flips to green on its own. Remove the `@Tag("known-defect")` at that point and it joins the regression gate. A tagged test must never be “fixed” by weakening its assertion to match current behaviour — that silently deletes the finding.

2.2 How a denied request is asserted

DEFECT-06 turns every authorisation denial into a 500. Asserting the correct 403 in all 11 controllers would have added roughly thirty further failures to the defect list, every one of them a restatement of the same root cause. The suite therefore separates two distinct claims:

- **The security decision** — that an unauthorised caller cannot act — is asserted in every controller, by verifying the service was never invoked. This holds today and will continue to hold after DEFECT-06 is fixed.
- **The status-code contract** — that the caller is correctly *told* 403 — is pinned once, by the six tagged tests in `DrugControllerSecurityTest` and `SaleControllerSecurityTest`.

Both facts are covered; neither is duplicated across the codebase. The reasoning is recorded in the Javadoc of each affected test class so the choice is not later mistaken for an oversight.

3. Defects

DEFECT-01**High****Rule 11 is not enforced where it matters — unverified controlled stock can be dispensed**

Area: SaleServiceImpl.createSale

Batch states its own contract explicitly: the service layer “must enforce that verifiedByPharmacist is required whenever drug.isControlledSubstance is true (Rule 11) before the batch counts as active/sellable stock”. BatchServiceImpl.verifyBatch correctly restricts *who* may verify, and the batch is correctly created unverified — but createSale never reads the field.

The result is that the verification step is advisory. A controlled-substance batch that no pharmacist has ever checked against its purchase order dispenses exactly like ordinary stock. The audit note still claims “single-pharmacist approval per Rule 3”, so the log records an assurance that was not obtained.

This is an omission rather than a broken check: there is no code to weaken, and no test previously existed to notice the gap.

Evidence

SaleServiceImpl.java:77-113 — the per-item loop reads batch.isExpired() and batch.getQuantityInStock(), but never batch.getVerifiedByPharmacist().

Reproducing test

```
SaleServiceImplTest > ControlledSubstanceVerification > rejectsUnverifiedControlledBatch
expected: BusinessException    actual: sale completed, stock decremented
```

Recommended fix

In the item loop, after the expiry check, reject any batch where drug.getIsControlledSubstance() is true and getVerifiedByPharmacist() is null, citing Rule 11. The companion test allowsVerifiedControlledBatch already passes and will guard against over-correcting into a block on verified stock.

DEFECT-06	High	Every authorisation denial is reported as a server error
------------------	-------------	---

Area: GlobalExceptionHandler / SecurityConfig

Spring Security raises `AuthorizationDeniedException` inside the controller invocation when a `@PreAuthorize` check fails. Because `GlobalExceptionHandler` declares a catch-all `@ExceptionHandler(Exception.class)`, Spring MVC's resolver claims that exception first — before `ExceptionHandlerFilter` further out in the chain ever sees it.

Three consequences follow. Clients receive **500 “An unexpected error occurred”** where the API contract promises 403, and cannot distinguish “you may not do this” from “the server is broken”. The server logs a full stack trace at ERROR for every routine permission check, which will bury genuine faults in alert noise. And `JwtAccessDeniedHandler` — written specifically to render the 403 body, and wired into `SecurityConfig` — is unreachable dead code.

To be clear about impact: the request *is* blocked, and the expanded controller tests described in §2.2 confirm that for all 11 controllers. This is a contract and observability defect, not a privilege-escalation one. It nonetheless affects every role rule in the system (Rules 2, 11, 12, 15, 16), which is why it is rated High.

Evidence

Observed response — PATCH `/api/v1/drugs/{id}/price` as PHARMACIST:

```
{
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred"
}
```

Server log: ERROR c.a.p.exception.GlobalExceptionHandler : Unhandled exception
org.springframework.security.authorization.AuthorizationDeniedException: Access Denied

Reproducing test

```
6 tests across DrugControllerSecurityTest and SaleControllerSecurityTest
  pharmacistMayNotUpdatePrice, technicianMayNotUpdatePrice, pharmacistMayNotReclassify,
  adminMayNotCreateDrug, technicianMayNotDispense, adminMayNotDispense
  expected: 403   actual: 500
```

Recommended fix

Add an explicit handler ahead of the catch-all:

```
@ExceptionHandler(AccessDeniedException.class)
public ResponseEntity<Object> handleAccessDenied(AccessDeniedException ex,
HttpServletRequest req) {
    return build(HttpStatus.FORBIDDEN, "Your role does not have permission to
perform this action", req);
}
```

`AuthorizationDeniedException` extends `AccessDeniedException`, so one handler covers both. Consider also narrowing the catch-all to exclude Spring Security's exception hierarchy, so no future security exception is swallowed the same way.

DEFECT-02 Medium Login discloses that a deactivated account exists

Area: AuthServiceImpl.login

The method carries a comment stating its intent plainly: the same generic message should be returned “whether the email is unknown or the password is wrong — don't let a login attempt reveal which accounts exist”. The implementation then checks `activeStatus` *before* verifying the password, and returns a distinct message for that case.

An attacker supplying any password therefore receives “This account has been deactivated” for a real former staff member and “Invalid email or password” for an address that was never registered — confirming the account exists without knowing any credential.

Scope is limited: only *deactivated* accounts are enumerable. An active account with a wrong password already returns the generic message correctly. The value to an attacker is nonetheless real, since former staff identify a valid email format and confirm the organisation's account naming.

Evidence

AuthServiceImpl.java:34-40 — `activeStatus` check precedes `passwordEncoder.matches()`.

Reproducing test

```
AuthServiceImplTest > Login > deactivatedAccountIsIndistinguishableFromUnknownEmail
  deactivated + wrong password : "This account has been deactivated"
  unknown email + same password : "Invalid email or password"
```

Recommended fix

Verify the password first, then check `activeStatus`, and return the generic “Invalid email or password” for a deactivated account. If operations need deactivated staff to understand why they cannot sign in, deliver that out of band rather than to an unauthenticated caller. Note that `JwtAuthenticationFilter` and `CustomUserDetails.isEnabled()` already reject deactivated accounts independently — both now covered by `SecurityComponentsTest` — so this reordering does not weaken enforcement.

DEFECT-03 Medium Pharmacist licence uniqueness is checked on create but not on update

Area: `StaffServiceImpl.updateStaff`

`createStaff` guards all three unique columns — email, phone and licence number. `updateStaff` guards only email and phone. Since the same endpoint can promote a Technician to Pharmacist and set a licence number, an update can claim a licence that already belongs to another pharmacist.

The database UNIQUE constraint on `staff.license_number` still prevents the duplicate row, so no data corruption occurs. But the failure arrives as a `DataIntegrityViolationException` at flush time, which no handler covers, so the user sees a 500 rather than the 409 the create path would give for the identical mistake.

Worth noting by contrast: the three reference-data services (Customer, Doctor, Supplier) all handle the equivalent case correctly, including the subtle sub-case of re-saving a record without changing its own natural key. Staff is the outlier.

Evidence

`StaffServiceImpl.java:72-77` — `existsByEmail` and `existsByPhoneNumber` are checked; `existsByLicenseNumber` is not, though it is checked at line 42-45 on create.

Reproducing test

```
StaffServiceImplTest > Administration > rejectsLicenceTakenByAnother
expected: DuplicateResourceException (409)   actual: no exception raised
```

Recommended fix

Mirror the create-path check, comparing against the current value so an unchanged licence is not rejected: if the requested licence is non-blank, differs from the staff member's existing licence, and `existsByLicenseNumber` returns true, throw `DuplicateResourceException`. `DirectoryServicesTest` demonstrates the pattern.

DEFECT-04 Medium Purchase-order delivery has no state guard and writes no audit entry

Area: PurchaseOrderServiceImpl.markDelivered

The method sets `status = Received` and stamps the delivery date unconditionally. A Cancelled order can therefore be resurrected into Received, and an already-Received order can have its recorded delivery date silently rewritten with no trace of the change.

Every other state transition in this codebase guards its starting state — `approvePrescription` and `rejectPrescription` both refuse to act on anything that is not Pending, and both write an audit entry. This method does neither, which reads as an oversight rather than a deliberate difference.

The missing audit entry compounds the first problem: back-dating a delivery is precisely the kind of change an audit log exists to record, and Rule 7 routes all such writes through `AuditLogService`.

Evidence

`PurchaseOrderServiceImpl.java:75-80` — four lines, no status check, no `auditLogService` call.

Reproducing test

```
PurchaseOrderServiceImplTest > Delivery > refusesToDeliverCancelledOrder  
  expected: BusinessException  actual: status became Received
```

Recommended fix

Reject the transition when the current status is Cancelled, and decide explicitly whether re-delivering an already-Received order is permitted (correcting a mistyped date is a plausible need — if so, allow it but audit it). Add an `ActionType.STOCK_UPDATED` audit entry either way. Note that the acting staff id is not currently passed to this method, so the signature will need it — `createPurchaseOrder` already takes one and can serve as the pattern.

DEFECT-05 Medium Malformed client input is reported as a server error

Area: GlobalExceptionHandler

The handler covers the domain exceptions and `@Valid` failures, but not the framework exceptions Spring raises when a request cannot be bound at all. Two were confirmed: a non-UUID path variable (`MethodArgumentTypeMismatchException`) and an unparseable enum in the body (`HttpMessageNotReadableException`). Both fall through to the catch-all and return 500.

The same root cause as DEFECT-06, with lower stakes: client mistakes are misreported as server faults, each one logging a stack trace at ERROR. The pattern is worth fixing once, comprehensively, rather than exception by exception.

A third instance is latent. `SaleServiceImpl` checks `existsByPrescriptionId` before inserting; if two dispense requests for the same prescription race, the database UNIQUE constraint correctly stops the second, but that arrives as an unhandled `DataIntegrityViolationException` — another 500. The double dispense is prevented; the reporting is wrong.

Evidence

```
GET /api/v1/drugs/not-a-uuid → 500 "An unexpected error occurred"
POST /api/v1/sales with "paymentMethod":"Bitcoin" → 500 "An unexpected error occurred"
```

Reproducing test

```
DrugControllerSecurityTest > ErrorMapping > malformedPathVariableIsBadRequest
  expected: 400   actual: 500
SaleControllerSecurityTest > Dispensing > unknownPaymentMethodRejected
  expected: 4xx   actual: 500
```

Recommended fix

Have `GlobalExceptionHandler` extend `ResponseEntityExceptionHandler`, which already maps the standard Spring MVC binding exceptions to 4xx, and override `handleExceptionInternal` to keep the existing response envelope. Add a handler for `DataIntegrityViolationException` → 409 to cover the race above and DEFECT-03's flush failure in one stroke.

4. Observations

These are not defects. They are decisions worth confirming, risks worth knowing about, and areas the current suite deliberately does not reach.

ID	Observation	Suggested action
OBS-01	The Flyway migrations and their CHECK constraints are never executed by the suite — it runs on H2 with Hibernate schema generation. Entity/schema drift would go unnoticed.	Add a Testcontainers-backed PostgreSQL integration test that runs the real migrations and exercises one write per table. This remains the single highest-value addition to the suite.
OBS-12	All 11 MapStruct mappers are unreferenced. Every service builds its responses with a hand-written <code>toResponse()</code> , so mapstruct-processor runs on every build generating implementations nothing calls.	Either adopt them consistently or delete them and drop the processor. Leaving both approaches in place invites a future change that updates only one.
OBS-02	Neither <code>BatchServiceImpl.createBatch</code> nor <code>markDelivered</code> writes an audit entry, though both change stock or its provenance. Rule 7 routes all such writes through <code>AuditLogService</code> .	Add STOCK_UPDATED entries, or document why receipt is deliberately excluded.

ID	Observation	Suggested action
OBS-03	No drug-catalog change is audited — including Admin-only price and controlled-status updates, the two most sensitive fields in the catalog (Rule 12).	Confirm whether this is intended. The DB CHECK constraint on <code>action_type</code> has no value for it, so a migration would be needed.
OBS-04	<code>toResponse()</code> issues a per-row query for line items on every list endpoint, plus lazy loads for each association. <code>getAllSales()</code> is $O(n)$ queries and will degrade as sales accumulate.	Use fetch joins or an <code>@EntityGraph</code> on the list queries. Add pagination to the unbounded list endpoints.
OBS-05	Read methods are not marked <code>@Transactional(readOnly = true)</code> ; lazy associations resolve via open-session-in-view, which Boot enables by default with a warning.	Annotate read paths and consider disabling OSIV once fetch strategies are explicit.
OBS-06	PATCH <code>/drugs/{id}</code> has replace semantics — omitting <code>genericName</code> blanks it rather than leaving it alone.	Behaviour is pinned by a passing test so any change is deliberate. Confirm it matches the API contract, or switch to merge semantics.
OBS-07	<code>getPurchaseOrdersAwaitingVerification</code> de-duplicates with <code>distinct()</code> , but <code>PurchaseOrder</code> does not override <code>equals/hashCode</code> — correctness depends on persistence-context identity, and the method is not <code>@Transactional</code> .	Works today via OSIV; de-duplicate by id explicitly to remove the dependency.
OBS-08	GET endpoints on sales carry no <code>@PreAuthorize</code> , so any authenticated staff member can read every sale and receipt.	Confirm this is intended. If receipts are considered patient data, restrict them.
OBS-11	The shared phone pattern constrains character set and length but not structure, so punctuation-only input with no digits at all passes validation. It guards Staff, Customer and Supplier alike.	Current behaviour is pinned by a passing test. Tighten the pattern to require a minimum digit count if stricter validation is wanted.
OBS-09	<code>application.properties</code> defines an unused <code>jwt.secret</code> alongside the bound <code>app.jwt.secret</code> .	Remove the stray property to avoid the impression that two secrets are in play.
OBS-10	The interaction and duplicate-therapy checks noted in <code>PrescriptionServiceImpl</code> are acknowledged in a comment as deferred.	Tracked here so the gap is visible in a QA record rather than only in a source comment.

5. Changes made to the project

All production code under `src/main/java` is untouched — this engagement reported defects rather than fixing them, so that each fix can be reviewed on its own merits against its failing test. The following build and test-scope changes were necessary to make the suite runnable.

File	Change	Why
<code>src/test/resources/application.properties</code>	Added — H2 in PostgreSQL mode, Hibernate create-drop, Flyway disabled, test JWT secret.	The main config resolves <code>DB_URL</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code> and <code>JWT_SECRET</code> from the environment with no fallback. Without this, no test could run — including the pre-existing <code>contextLoads</code> , which was failing on any clean checkout. The file must sit under <code>src/test/resources</code> , not in the Java source tree, or it never reaches the test classpath.

File	Change	Why
pom.xml	Added test dependencies spring-boot-starter-webmvc-test and spring-boot-starter-security-test.	Spring Boot 4 split the test slices out of spring-boot-starter-test into per-module starters. @WebMvcTest lives in the first; the second carries the configuration that applies Spring Security to the auto-configured MockMvc, without which @WithMockUser never reaches the filter chain and every authenticated slice test returns 401. Both were already anticipated in a commented-out block in the pom.
pom.xml	Configured maven-surefire-plugin with <excludedGroups>known-defect</excludedGroups>, overridable per run.	Keeps the regression gate green while preserving the defect reproductions as executable, runnable evidence rather than prose.

A note on the environment

The suite was built and run on JDK 26 against the project's declared Java 21 release target, using the bundled Maven wrapper. All 326 regression tests pass and all 12 defect reproductions fail as documented, so the results in this report are reproducible from a clean checkout with no database, no network and no environment variables. The first build requires internet access to download dependencies.

6. Recommendations

In priority order:

- **Fix DEFECT-06 first.** It is roughly four lines, it restores the intended 403 contract across every protected endpoint, and it makes ERROR-level logs meaningful again — which matters before any other defect is investigated in a running environment.
- **Then DEFECT-01.** It is the only finding where a business rule protecting controlled substances is simply absent, and the audit log currently asserts an assurance that was never obtained.
- **Fix DEFECT-05 structurally** by extending ResponseEntityExceptionHandler and adding a DataIntegrityViolationExceptionHandler handler. That one change also cleans up DEFECT-03's flush failure and the latent double-dispense race.
- **Add a Testcontainers PostgreSQL integration test (OBS-01).** It closes the largest remaining blind spot: the migrations and CHECK constraints that the whole persistence layer depends on are currently unexercised.
- **Resolve the MapStruct question (OBS-12).** Eleven unreferenced classes and an annotation processor run on every build for no effect; adopt or delete.
- **Wire the suite into CI** so mvn test gates merges, and schedule the known-defect run to report separately — it doubles as a burn-down chart for this report.
- **Resolve the open questions in §4** — particularly OBS-03 and OBS-08, where the correct behaviour depends on a policy decision rather than on anything visible in the code.

Overall assessment. This is a sound codebase with a clear separation of concerns and business rules kept in the service layer where they can be tested. The defects found are concentrated in two places: one unimplemented rule, and a single over-broad exception handler whose effects reach every endpoint. Neither requires structural change. With the six fixes applied and the accompanying tests untagged, the suite gives 338 passing checks across every service, controller and security component in the system.